A Tale of Two Fashions: An Empirical Study on the Performance of Native Apps and Web Apps on Android

Yun Ma, *Student Member, IEEE*, Xuanzhe Liu, *Member, IEEE*, Yi Liu, Yunxin Liu, *Member, IEEE*, and Gang Huang, *Member, IEEE*

Abstract—The prevalent smartphones have become the major entrance to accessing services on the Internet. On smartphones, users can have two options as the clients, i.e., native apps and Web apps. There have been several debates about native apps and Web apps. However, major service providers such as `Google`, `Amazon`, and `Facebook` provide both native apps and Web apps to end-users. Essentially, the performance differences between these two types of apps haven't been addressed. Indeed, the performance differences make non-trivial impacts on apps development, deployment, and distribution. In this article, we conduct a measurement study on the performance of native apps and Web apps on Android smartphones. Specifically, we want to explore given the same functionalities, do Web apps always perform poorly compared to native apps. We select 328 services from some popular providers, covering various domains such as e-commerce, map, social networking, and entertainment. With HTTP-level trace analysis, we demystify the workflows on how native apps and Web apps deliver services on mobile devices, respectively. Then, we characterize the performance differences between native apps and Web apps with the metrics including the number of requests, response time, data drain, and energy consumption. We find that the performance of Web apps is better than native apps in more than 31 percent cases. Our derived knowledge can suggest some recommendations to improve the performance for mobile apps.

Index Terms—Services, mobile applications, web, smartphones

1 Introduction

After the announcement of iPhone in 2007, the sales and popularity of mobile devices such as smartphones and tablet computers keep tremendously increasing in the past decade. Mobile devices have become the main entrance to the Web services. Mobile users use mobile applications (a.k.a., apps) to access these services [1], e.g., weather query by *Yahoo!*, Web search by *Google*, shopping by *Amazon*, and social networking by *Facebook*.

There are two types of mobile apps from which users can access Web services, i.e., native apps and Web apps [2]. Native apps are developed in imperative languages (e.g., Java and Objective-C) and released as software packages that can be published on and downloaded from online app stores. Web apps are developed with Web technologies (such as HTML, JavaScript and CSS) and *directly* accessed via browsers. Recently, a medium app called "hybrid app" can be viewed as a special type between native apps and Web apps. In practice, hybrid apps are packaged and released as native apps, and most of their key features are developed by Web technologies [3]. The behavior of hybrid apps is a combination of native apps and Web apps.

Conceptually, both native apps and Web apps can perform specific tasks and provide features to the end-users. For example, we can view the timeline through either the native app or the Web app provided by *Facebook*. Although these two types of apps can provide similar or even the same functionalities, there have been several debates between them [2]. From a technical perspective, the native apps are claimed to better leverage hardware capabilities of devices and are supposed to provide better performance. However, developing native apps requires a lot of efforts to deal with various platform-specific features. For example, iOS apps are developed in Objective-C or Swift, and Android apps are developed in Java, which increases their development and maintenance cost. Additionally, the package-based software release model results in various versions of native apps, while the end-users have complaints as they have to always keep updating to the latest versions [4]. In contrast, the Web apps are developed by standard HTML, JavaScript, and CSS, and can be opened in popular Web browsers such as Chrome, Safari, and Firefox on any OS. Such a feature reduces the development and maintenance cost. Users can access the "*always up-to-date*" Web apps without having to manually update them. Our recent study over millions of Android users shows that the Web browser is still the major app through which Android users access Web

- *Y. Ma, X. Liu, Y. Liu, and G. Huang, are with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education, Beijing 100871, China. E-mail: {mayun, liuxuanzhe, liuyi14, hg}@pku.edu.cn.*
- *Y. Liu is with Microsoft Research, Beijing 100080, China. E-mail: yunxin.liu@microsoft.com.*

Manuscript received 5 Jan. 2017; revised 11 Sept. 2017; accepted 19 Sept. 2017. Date of publication 26 Sept. 2017; date of current version 2 Apr. 2018. (Corresponding author: Xuanzhe Liu.)
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TMC.2017.2756633

1536-1233 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

services [5]. Although Web apps are reported to perform more slowly than native apps and the user experience is not satisfactory enough [2], a recent study found that 153 out of the top 200 websites in Alexa[1] still provide the Web apps that are optimized for mobile platforms [6].

With the advances of HTML5, Web apps are supposed to have comparable performance and experiences of native apps. However, to the best of our knowledge, very few previous efforts demonstrate whether and to what extent Web apps can be comparable to native apps. In this article, we are interested in answering a central problem: *given the same or similar functionality, do Web apps always perform more poorly than the native apps?* The answer to this question is nontrivial and meaningful in two folds. First, if the performance of Web apps is close to that of native apps, service providers can reduce the maintenance cost of multiple versions, and end-users can get rid of the updates. Second, even if the performance of Web apps is poor compared to that of native apps, demystifying the underlying technical causes can provide insightful knowledge for the service providers and Web app developers, who can then find potential solutions to optimize the Web apps.

In this article, we break down the central problem by studying three uncovered problems: (1) how native apps and Web apps access Web services on mobile devices; (2) to what extent the performance of native apps and Web apps varies; (3) what kind of knowledge developers can leverage to develop and optimize the apps. To this end, we consider four quantitative metrics to characterize the performance: the number of HTTP(S) requests, traffic volume, response time, and energy. We select 328 distinct services from 12 popular service providers, i.e., *Facebook, Amazon Shopping, Google Map, Baidu Map, Sina Weibo, Dianping, Gewala Movie, Jd, Maoyan Movie, Meituan, Zhihu*, and *Douban*, which develop both native apps and Web apps with the same or similar functionalities for mobile devices. We choose Android as the target platform of native apps as Android is the most popular mobile OS nowadays. In the rest of this article, the native apps refer to Android apps if not specified. By using two Android smartphones with the same device model under the same network condition, we conduct our measurement study and find that in about 31 percent of the situations, Web apps perform much better than the native apps, when providing the same functionality. Based on the analysis of various aspects such as network protocol, third-party libraries, and cache efficiency, we suggest some possible recommendations that may help to make the performance of Web apps comparable to that of native apps.

This article is the extension of our previous ICWS 2015 paper [7] that made the first study of performance differences of Web services delivered through native apps and Web apps. Compared to the conference version, this article significantly extends the scale of Web services (from 52 to 328) and adds a new metric of energy consumption. Besides obtaining the almost consistent findings of the conference version, our results further demonstrate some imperfect designs that can lead to unsatisfactory performance and user experiences on smartphones. We also provide some useful knowledge and suggestions to service providers, Web/native app developers, and end-users. The main contributions made in this article can be summarized as follows:

- We establish an experimental platform[2] to collect the network-level traces of requesting Web services on smartphones, and propose a trace-based approach to characterizing the usage of Web services of native apps and Web apps, respectively. Based on the trace analysis, we demystify how Web services are used from an architectural style perspective.
- We design a series of experiments to measure the performance of native apps and Web apps, given by the same functionality. Our metrics include the number of HTTP(S) requests, response time, traffic volume, and energy consumption. In about 31 percent of the situations, Web apps perform much better than native apps. We then analyze the possible reasons that can lead to such differences.
- We draw some implications from our empirical study and propose recommendations on how to improve the performance of mobile apps from various aspects such as network protocols, cache, and API semantics. In particular, we attempt to make Web apps provide comparable performance levels and user experiences to native apps.

The remainder of this article is organized as follows. Section 2 introduces some background knowledge. Section 3 describes the measurement methodology and how we conduct our experiments. Section 4 anatomizes the architecture of native apps and Web apps on using Web services, respectively. Section 5 analyzes the performance differences of native apps and Web apps, given the same functionality. Section 6 summarizes our findings according to the architectural differences. Section 7 provides implications to service providers and developers. Section 8 discusses the threats to validity of our approach. Section 9 presents related work, and Section 10 concludes this article with future outlook.

2 Background

Traditionally, there are two types of apps to deliver Web services on Smartphones.

• *Native apps*. A native app is developed for a specific OS platform, i.e., iOS, Android, or BlackBerry. Native apps are usually developed in an imperative programming language. For example, Android apps are developed in Java, and iOS apps are developed in Objective-C or Swift. Native apps can take advantage of the particular features of mobile devices, such as various sensors. However, most parts of native apps cannot be executed across different platforms, whereby developers have to maintain various versions for different platforms. End-users usually need to download native apps from app stores, such as *Apple AppStore*, *Google Play*, and so on. In addition, end-users have to manually keep updating of native apps.

• *Web apps*. A Web app is developed for running within Web browsers or browser-based runtime such as WebKit and Gecko. It is developed by Web technologies, such as

1. http://www.alexa.com

2. We have released the whole dataset along with the work published. Please contact the corresponding author, Xuanzhe Liu, via liuxuanzhe@pku.edu.cn, for more details.

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

HTML5, CSS3, and JavaScript. End-users enter the URL in the address bar or click through the hyperlinks, and then navigate to the latest version of Web apps. Web apps have advantages in terms of cross-platform compatibility, or the so-called "*write once, run anywhere*" feature. With the advances of HTML5, Web apps are claimed to provide comparable user experiences to native apps. However, due to the limitation of Web technologies, standard Web apps cannot take full advantage of the devices' hardware capabilities, and their performance is often complained to be poorer than native apps in practice. Sometimes, users can perceive quite obvious latency when loading a Web app.

There is another type of apps, called *Hybrid apps*, that could be regarded as an intermediate type between native apps and Web apps. Similar to the native apps, the hybrid apps also need to be downloaded from app stores. Although the hybrid apps wear the "shell" of native apps, the core logic of hybrid apps heavily relies on Web technologies. As a result, hybrid apps can support cross-platform portability and reduce development costs. Many tools and frameworks could help to develop hybrid apps, such as Apache Cordova [8], manifoldJS [9], and React Native [10]. The behavior of hybrid apps is a combination of native apps and Web apps. In order to make a clear comparison, in this article, we do not take hybrid apps into account and leave them as future work.

Nowadays, service providers deliver their functionalities via Web services. Web services were initially proposed as a way that a software system can support interoperable machine-to-machine interactions over a network [11]. Commonly, there are two main types of Web services, the SOAP-based ones and the RESTful ones. There have been a lot of debates on comparing these two types of Web services [12]. In the context of mobile computing, it is observed that SOAP-based Web services are hardly applied due to the non-negligible performance overheads and restrained computation resources on mobile devices. The RESTful Web services are more adequate to the heterogeneous mobile networks and resources-limited mobile devices. Many mainstream service providers, including *Facebook*, *Google*, and *Amazon*, have deprecated or abandoned SOAP-based interfaces. RESTful Web services allow developers to use HTTP methods explicitly in a way that follows the protocol as defined by RFC 2616 [13]. Four basic HTTP methods can operate resources deployed on the Web server: (1) `GET` refers to retrieve a resource from the server; (2) **`POST`** refers to create a sub-resource on the server; (3) `PUT` refers to create a new resource or update the state of an existing resource on the server; (4) `DELETE` refers to remove or delete a resource on the server.

3 Measurement Methodology

In this section, we describe the rationale of our trace-based measurement methodology. We provide details of how our empirical study is designed, including the hardware and software setup, metrics and their definitions, and the measurement workflow. We then propose three research questions that are to be answered by our empirical study.

3.1 The Dataset

Trace-based performance measurement is widely used in analyzing network performance [14], [15], [16], [17]. We make a trace-based analysis to anatomize the workflows of native apps and Web apps by requesting the same functionality. We choose apps from categories that heavily rely on network, including social networking, e-commerce, entertainment, and so on. For each category, we select some popular service providers who provide both native apps and Web apps. Finally, our dataset consists of native apps and Web apps from 12 popular service providers, including *Facebook, Amazon Shopping, Google Map, Baidu Map, Sina Weibo, Dianping, Gewala Movie, Jd, Maoyan Movie, Meituan, Zhihu*, and *Douban*, as shown in Table 1. It can be observed that the chosen service providers cover both international and local (China) ones. Such a diverse and representative dataset can promise our measurements with little bias. For each service provider, we choose the features that have the same user-perceived functionalities between the native app and Web app. To this end, we manually examine every single feature to ensure that the displayed contents on the UI are exactly the same with the two apps. Finally, we choose 260 features involving 328 distinct Web services[3]. We list the number of selected features and Web services by each provider in Table 1.

TABLE 1
Selected Apps of Targeted Web Services

APP Name	Description	# of Features	# of Services
Facebook	Social Networking	31	33
Sina Weibo	Social Networking	19	25
Amazon	e-Commerce	29	34
Jd	e-Commerce	32	39
Google Map	Map	16	22
Baidu Map	Map	16	29
Dianping	Lifestyle	22	31
Douban movie	Entertainment	19	23
Gewala movie	Entertainment	18	25
Maoyan movie	Entertainment	14	17
Meituan	Groupon Shopping	20	24
Zhihu	Online Q& A	24	26

3.2 Definitions and Metrics

We first define some terms used to describe our approach.

Definition 1 (Service). *A service is an atomic functionality implemented by the service providers. For example, adding goods to cart and deleting goods from cart in Amazon Shopping should be viewed as a service.*

Definition 2 (Feature). *A feature is a functionality that may request a specific (set of) services, such as login, checking out the shopping cart, and so on. A feature can comprise more than one service, or, a composite service. For example, when users log on the Facebook, the client program needs to request three services including retrieving the user profile, updating the status of friends, and checking the timeline.*

Definition 3 (Trace). *A trace is a sequence of HTTP/HTTPS requests and responses when using a feature. Fig. 1 shows three traces of Amazon Shopping, including the features of searching keywords, adding goods to cart, and deleting goods from cart, respectively.*

3. We have released the dataset on our website at http://sei.pku.edu.cn/~mayun11/NativeWeb, including the details of features and measurement results

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

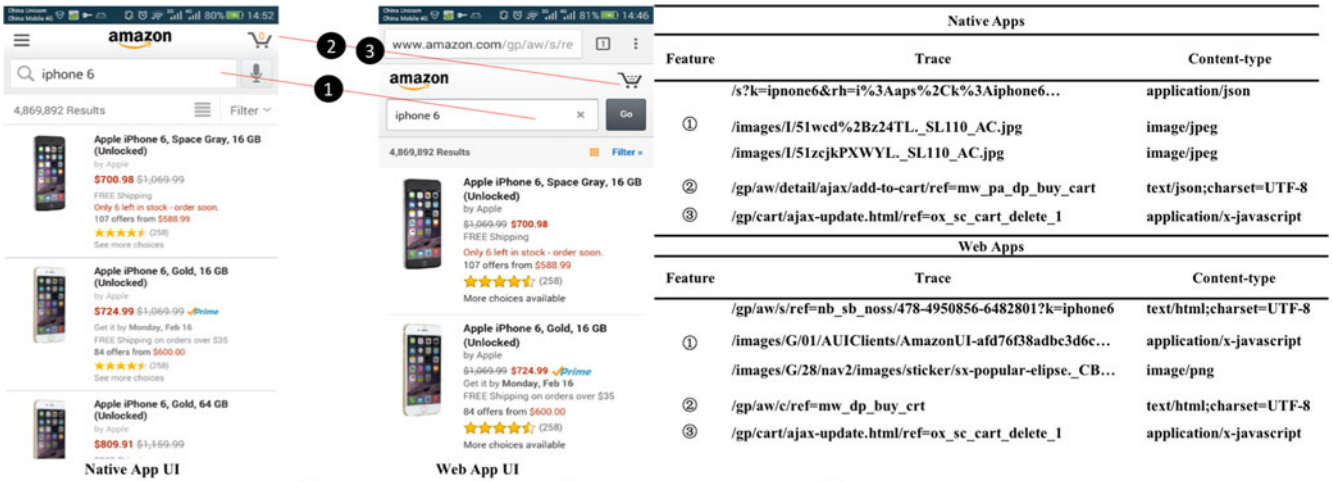


Fig. 1. Trace logs of the native app and the web app of *amazon shopping* generated by the same feature.

Based on the definitions above, we then define the metrics used to measure the performance in our study. Learning experiences from previous work [15], [16], we focus on four metrics.

Metric 1 (Number of HTTP(S) Requests). *Refers to the number of HTTP(S) connections sent by the app when using a feature*. It can reflect how many HTTP resources are needed to accomplish a specific feature. For example, for the feature of *adding goods to cart* in *Amazon Shopping*, there is only one request "*/aw/detail/ajax/add-to-cart/*" sent from the native app, as show in Fig. 1.

Metric 2 (Traffic Volume). *Means the total data volume involved in a trace, including the size of bytes sent and received via the app.*

Metric 3 (Response Time). *Is the interval from the time when the first network request is issued to the time when the last response is received in a trace*. The response time is not equivalent to the user-perceived time when using a feature. Instead, it comprehensively indicates how long a feature can be accomplished at the system level. As shown in Fig. 1, the response time of *searching keywords* feature means the period between the start time of the first request "*s/k=iphone...*" and the end time of the last response "*/images/...*"

Metric 4 (Energy Consumption). *Is the total amount of energy consumed from the time of the first issued network request to the time of the last received response in a trace*. We employ a physical *Monsoon Power Monitor*[4] to record the power consumption of the connected smartphones and compute the energy consumption when requesting each service. By applying the *V-Edge* energy model proposed in our previous work [18], we can accurately compute the energy consumption as follows:

$$energy_consumption = \int_{start_time}^{end_time} V\text{-}Edge(x) \qquad (1)$$

where the *start_time* means the start time of a trace, *end_time* means the end time of a trace, and x means the time between *start_time* and *end_time* used in the *V-Edge* power model. Due to page limit, the details of *V-Edge* can be referred from previous paper [18].

3.3 Experiment Setup

Next, we introduce the infrastructure of our experimental platforms and how we conduct our experiments.

- *Hardware.* We use two smartphones of the same device model. The selected device model is Samsung Galaxy S4 with 2 GB RAM and Android 4.2 OS (directly downloaded from Google rather than using the manufacturer customized ROM). The two smartphones are connected to the same Wi-Fi network, and are "rooted" with the system-level privilege. We uninstall all other pre-installed apps, stop background activities such as OS auto-update and auto-sync services. In this way, we can prevent other apps from connecting the network and significantly reduce the possible noise. To inspect the details of network activities, we use the popular Wireshark tool [19] to inspect the traffic to/from our smartphones. Although the WireShark tool can possibly impact the results of energy measurement, previous studies show the impact is too marginal and could be neglected [18].
- *Software.* For each service provider, we install and access the native app on one smartphone, and open the Web app within Chrome Browser of version 40.0.2214.89 on the other smartphone. We manually use the native and Web app to access each selected feature. For each feature, we repeat three times to avoid the biased measurements. In order to make the comparison fair enough, we concurrently access the same feature of both types of apps with two people using the two smartphones simultaneously.

3.4 Tracing Web Services Usage

We deploy a proxy server which runs both the WireShark and the popular HTTP/HTTPS-level proxy called *Fiddler*[5] to intercept all network traffic between the smartphones and the remote Web services. In this way, we can explore the network details more accurately by investigating the trace logs from both smartphones and the proxy server.

4. https://www.msoon.com/LabEquipment/PowerMonitor/

5. http://www.telerik.com/fiddler

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

Then, we compare the network traces of native apps and Web apps, given the same feature. For example, we compare the traces of `searching_a_person` feature offered by *Facebook*, by using the same keyword query on the Web app and the native app.

As the cache mechanism is widely used in both native apps and Web apps [20], we also request each feature of the targeted Web services in two modes, *Without Cache* and *With Cache*.

- *Without Cache*. To disable the cache of native apps, we manually uninstall a native app, and remove all files in its folder. Then we re-install and revisit the app for the same feature. To disable the cache of Web apps, we manually clear the browser cache of a Web app and reload the app. At each time we record the network traces and energy information, and calculate the number of HTTP(S) requests, traffic volume, response time, and energy consumption.
- *With Cache*. We test the same feature of the two types of apps, with the default cache mechanisms turned on. We record the same information and calculate the metrics as well.

3.5 Research Questions Statement

Based on the dataset and hardware/software configurations, our empirical study aims to measure how the native apps and Web apps perform, given the same functionality. More specifically, we aim to answer the following research questions:

- *RQ1*: *How do native apps and Web apps access Web services on smartphones, respectively?*
- *RQ2*: *How do native apps and Web apps perform differently when providing the same functionality?*
- *RQ3*: *What kind of knowledge can developers leverage to optimize the apps?*

4 Architecture Anatomy

We first focus on *RQ1*, i.e., *how do native apps and Web apps access Web services, respectively?* Answering such a question can help us have the basic insight of how the two types of apps request a Web service. Based on such knowledge, we can then make in-depth performance analysis.

4.1 Web Service Identification

For better illustration, we take a popular service provider, the *Amazon Shopping*, as our illustrating example. Fig. 1 shows some traces of *Amazon Shopping* on both the native app (at left side) and the Web app (at right side). When users fill in the information and click the button, some consequent requests and respective responses will be triggered. We list traces of three features, i.e., the `search_keywords` feature, the `add_goods_to_cart` feature, and the `delete_goods_from_cart` feature.

All of the 328 Web services in our dataset, are provided in the RESTful fashion. Such an observation evidences that SOAP-based Web services are not widely adopted in the context of mobile Internet. It can be found that the contents of service responses are formatted with JSON in both native apps and Web apps. Although these two types of apps have

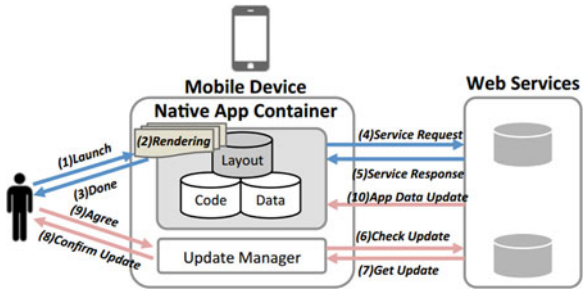


Fig. 2. The architecture of native app.

similar user interfaces, the requested URLs of the same feature are entirely different. For example, when we add a good to the cart, the native app sends a request to the URL of "*/aw/detail/ajax/add-to-cart/*", while the Web app sends a request to the URL of "*/aw/c/*". By exploring all of the 328 Web services in our dataset, it is quite interesting that the target URLs of Web services requested by native apps and Web apps are always different for the same feature. It implies that Web service providers maintain different interfaces for native apps and Web apps. In addition, it is interesting to find that native apps and Web apps may use different IP addresses to access the same services. Therefore, we can infer that the server-side service implementations of the same feature are separately deployed for native apps and Web apps, although the two types of apps still access the same back-end database.

Based on the traces, we then try to figure out the workflow of how native apps and Web apps request a Web service, respectively. We examine the response of each request and infer its function in the workflow from the header fields and some keywords in the content such as update, version, etc.

4.2 Architecture of Native Apps on Using Web Services

Fig. 2 shows the architecture on how a native app uses Web services. Once a native app is installed on the device, the native code, including the data parsing, network management, and layout rendering, are maintained locally on the smartphone. In addition, some resources such as icons and images can be configured by the developers and locally stored. When a user launches the native app and clicks some buttons to issue interaction requests, the processing logics of native code are executed. The native code may trigger requests to the target Web services, and send synchronous/asynchronous requests to fetch data. When requesting Web services, the native apps come up with lots of preloaded elements and fetch only the "required" data from the Web server rather than the contents of the entire application. The native app can create a new thread to issue an asynchronous request when a resource is required. When the data is downloaded, the native code can parse the data, process the data with the preloaded logic, and re-render the new user interface.

4.3 Architecture of Web Apps on Using Web Services

In contrast, the Web apps are accessed in the smartphone's Web browser such as Chrome, FireFox, and Safari, and update themselves without user intervention. Web apps are built by standard technologies such as HTML5, CSS, JavaScript, and so on. Fig. 3 shows the architectural workflow

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

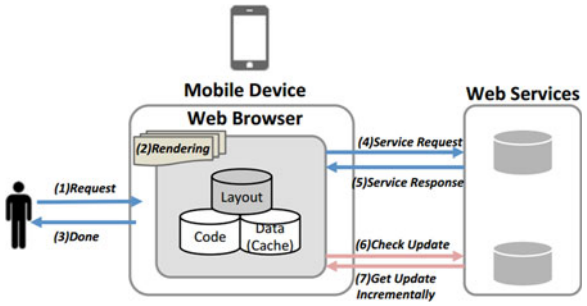


Fig. 3. The architecture of mobile web app.

on how a Web app requests a Web service. The Web app has to download all resources such as page files, images, JavaScript files, and CSS files to render the user interface in the mobile browser. When a user first loads the Web app, the browser initiates a request to fetch the structural information of the main page. Then the browser interprets the HTML, JavaScript, and CSS files to build up user interfaces. There are two forms of user interactions over the Web apps. The first one is to input some text fields or submit some forms, then the Web apps jump from one page to another page. The other one is that the Web app generates an AJAX (Asynchronous JavaScript and XML) request, and deals with the callback methods without page jumping. As the Web apps fetch the latest resources of a Web service, the user is always displayed by the latest version of the app.

4.4 Architectural Differences

Based on the above analysis, we can have an observation that native apps and Web apps have quite different workflows and behaviors of requesting Web services. Besides the different implementation languages and different RESTful API semantics, two differences need to be mentioned.

The first difference is the cache management of resources. Compared to the Web apps that always present the up-to-date version, the native apps are updated manually when the users explicitly launch the *Update Manager* that is responsible for checking updates and downloading the latest version. Rather than relying on the browsers that manage the resources to be fetched from the remote server, *developers of native apps can programmatically configure the need-to-update resources*. As a result, the "stable" resources such as icons and images are permanently stored in the local storage, and the update manager fetches only the "delta" resources. Such a mechanism is quite effective to better reuse the already-downloaded resources, and thus reduces the unnecessary data transfer. In contrast, the cache management of Web apps usually depends on the browser and some coarse-grained policies [6].

The other difference is the network resources management, which can be inferred from trace logs of WireShark and Fiddler. We employ the existing techniques [16] to analyze the traces of WireShark and synthesize the TCP connections. In fact, both native apps and Web apps need to setup TCP connections to the RESTful Web services. It is interesting to find that, due to the constraints of browsers such as Chrome and FireFox [15], the Web apps usually occupy at most 6 concurrent TCP connections per domain. In contrast, there are more concurrent TCP connections in native apps, as the developers can configure the number of connections by themselves. More specifically, the widely-adopted `AndroidHttpClient` API and other third-party libraries such as `okhttp` [21] and `Volley` [22] usually support a developer-defined number of TCP connections.

These two reasons can preliminarily reflect some fundamental architectural differences between native apps and Web apps. In the following empirical studies, we move on to exploring the performance differences and correlate such differences to the architectural style.

TABLE 2
The Percentage of Genres Where Web Apps Beat Native Apps

Operation	HTTP(S) Requests	Traffic Volume	Response Time	Energy
PUT&POST	22.61%	28.26%	36.96%	36.96%
DELETE	20.24%	23.81%	45.24%	42.86%
GET	42.97%	24.22%	37.50%	41.41%

5 Measurement Results

In total, we have 260 distinct features that consist of 328 Web services: 66 `PUT` & `POST` operations, 64 `DELETE` operations, and 198 `GET` operations. As we focus on 4 metrics and evaluate each metric with/without enabling the cache on two types of apps, finally we have 260*4*2=2,080 genres of data.

We now focus on *RQ2*, i.e., *how do native apps and Web apps perform differently when providing the same functionality?* Given the same features, it is intuitively assumed that the performance of Web apps is usually less satisfactory than native apps. However, it is surprising to find a substantial percentage (about 31 percent) of exception where the Web apps perform better than the native apps, i.e., in *645* out of our *2,080* genres, as shown in Table 2. It implies that there are some possible reasons to make the Web apps have comparable performance with native apps. *Due to space limit, this article reports only the cases where Web apps beat the corresponding native apps.* The full experimental evaluation results can be found at our lab server via http://sei.pku.edu.cn/~mayun11/NativeWeb.

Web services of different RESTful operations represent different kinds of functionalities and their behaviors are potentially different. In order to clearly investigate different reasons leading to the different performance of native and Web apps, we categorize all the tested features into three groups according to the type of RESTful operations, i.e., `GET`, `DELETE`, `PUT` & `POST`. We aggregate the `PUT` and `POST` operations into one group, as they both result in resources updating on the server. We then report and analyze the performance of each category, respectively.

5.1 PUT&POST

In the semantics of RESTful Web services, the `PUT` refers to the request that creates a new resource or modifies the existing resource, and the `POST` refers to the request that sends a representation of a new data entity to the server so that it will be stored as a new subordinate of the resource. We choose seven typical features that need `PUT&POST` operations where the Web apps provide the comparable or even better performance than native apps: the `post_timeline` feature in *Facebook*, the `add_comment` feature in *DianPing*,

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

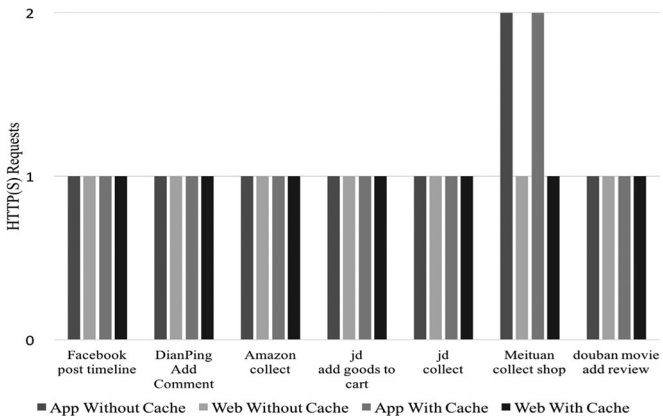


Fig. 4. Number of requests in PUT&POST.

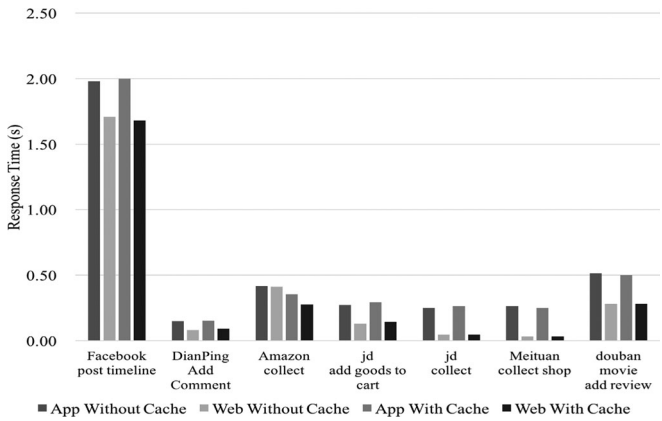


Fig. 6. Response time in PUT&POST.

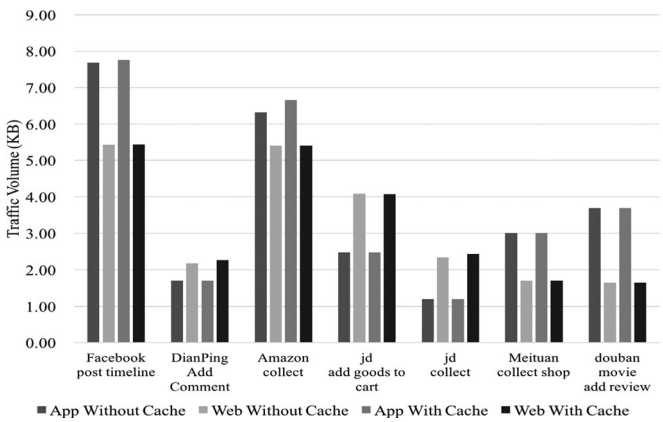


Fig. 5. Traffic volume in PUT&POST.

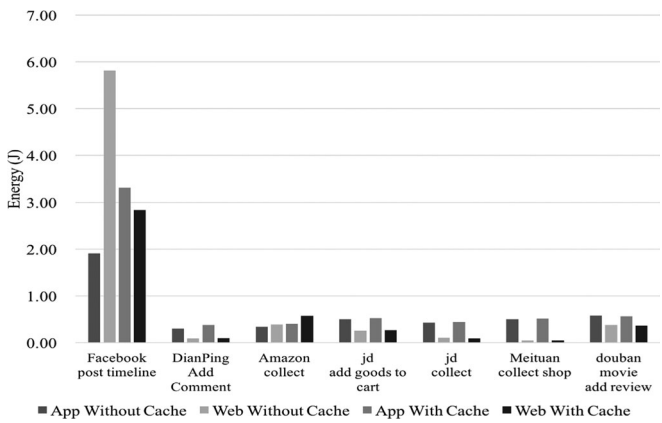


Fig. 7. Energy consumption in PUT&POST.

the `collect_goods` feature in *Amazon Shopping*, the `add_goods_to_cart` feature and `collect_goods` feature in *Jd*, the `collect_shop` feature in *Meituan*, and the `add_review` feature in *Douban*. Figs. 4, 5, 6, and Fig. 7 show the number of HTTP(S) requests, the traffic volume, the response time, and the energy consumption of the above `PUT&POST` operations, respectively.

Number of Requests. For `PUT&POST` operations, we find that the number of HTTP(S) requests does not vary very much. Both the native app and Web app initiate almost the same number of requests with or without cache. Such an observation is not surprising as the `PUT&POST` operations usually do not need local access. However, some exceptions are observed. For example, for the `collect_shop` feature of *Meituan*, the native app initiates more network requests. The reason is that such a feature needs to get the latest list of all collected shops after the `POST` operation is finished, and triggers one additional request. However, in fact such redundant transfer is not necessary and the two requests can be merged.

Traffic Volume. Unlike the number of HTTP(S) requests, the traffic volume differs between the two types of apps. Generally, the Web apps need more traffic volume than native apps for the same `PUT&POST` operation. The reason is that Web apps usually send more information in `POST` & `PUT`, such as more cookies and longer User-Agent, to the servers. However, some exceptions need to be mentioned. For the *Facebook's* `post_timeline` feature, the Web app saves traffic than the native app, no matter the cache is enabled or not (29.3 percent saving and 29.9 percent saving, respectively). It is because that the native app refreshes the whole timeline after the `POST` action is triggered, instead of refreshing only the latest status. Other exceptions can be found in the `add_review` feature of *Douban* (about 55.4 percent saving), the `collect` feature of *Amazon Shopping* (about 14.5 percent saving), and the `collect_shop` feature of *Meituan* (about 43.6 percent saving). In addition, we can also find that the cache mechanism marginally works in the `PUT&POST` operations.

Response Time. In most cases, more traffic volume means longer response time. However, we can find that sometimes the Web app has shorter response time than the native one. For example, the `add_goods_to_cart` feature and the `collect` feature of *Jd*'s Web app consume more traffic volume than those of *Jd*'s native app, but the Web app's response time is shorter (more than 50 percent faster) than the native app's.

Energy Consumption. Overall, the Web apps perform more poorly than the native apps in terms of energy consumption. However, the preceding mentioned features with shorter response time for Web apps still save energy compared to native apps. For example, the native apps need to record the states of the device, write to the local disk, and communicate with the remote servers. The I/O and network transmission can result in more energy consumption than Web apps.

5.2 DELETE

In the semantics of RESTful Web services, `DELETE` refers to the request that deletes an existing resource on the Web server. We show seven typical features that correspond to

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

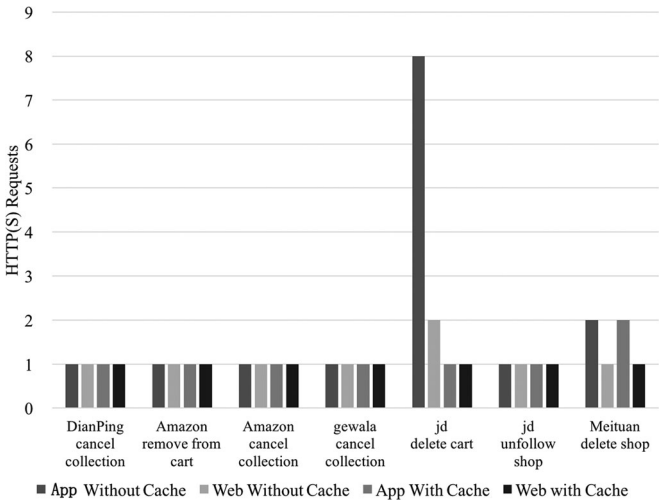


Fig. 8. Number of requests in DELETE.

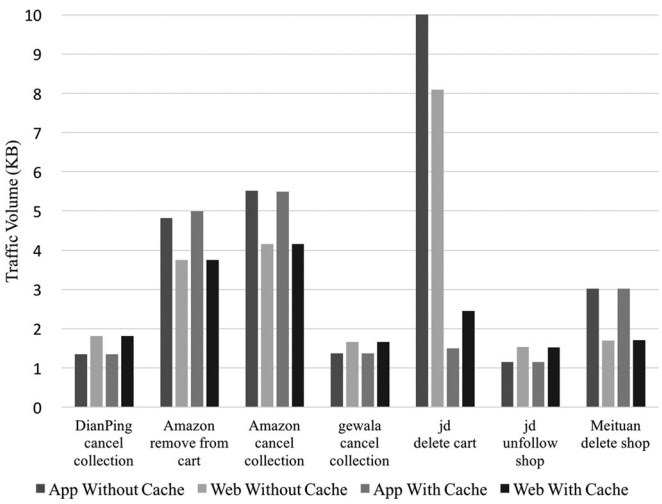


Fig. 9. Traffic volume in DELETE.

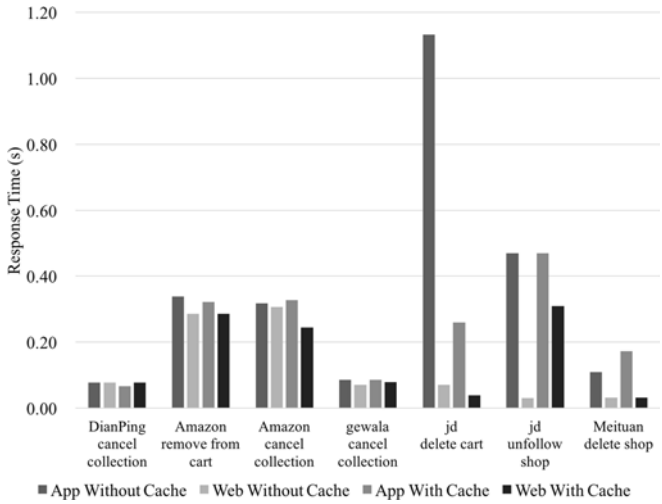


Fig. 10. Response time in DELETE.

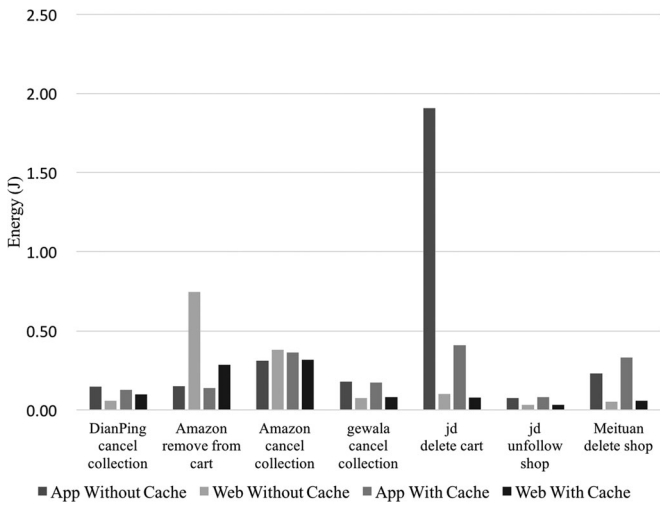


Fig. 11. Energy consumption in DELETE.

`DELETE` operations, i.e., the `cancel_collection` feature in *DianPing*, the `remove_goods_from_cart` feature and the `cancel_collection` feature in *Amazon Shopping*, the `cancel_collection` feature in *Gewala movie*, the `delete_cart` feature and `unfollow_shop` feature in *Jd*, and the `delete_shop` feature in *Meituan*. Figs. 8, 9, 10, and Fig. 11 show the number of HTTP(S) requests, the traffic volume, the response time, and the energy consumption of `DELETE` operations, respectively.

Number of Requests. In most cases, both the native apps and Web apps initiate the same number of requests to convey the deletion command. In term of the `delete_cart` feature in *Jd*, the native app initiates an additional request to fetch some recommended items for customers after clearing the shopping cart. In terms of the `delete_shop` feature in *Meituan*, the native app initiates a new request to fetch all collected shops after deleting a shop. These additional requests are not sent in the Web apps. To reduce the cost of network resources, we shall recommend that the service provider should merge these requests.

Traffic Volume. Similar to the `PUT&POST` operations, the native apps of *Meituan* and *Amazon Shopping* consume more traffic volume than the Web app. The native app of *Meituan* fetches the latest collected shops list after deleting a shop. The native app of *Amazon Shopping* transfers data by HTTPS, while the Web app transfers data by HTTP. We also find that the native app of *Amazon Shopping* adds extra cookie fields in the request header, such as device info, application version, etc. For the feature of `delete_cart` in *Jd*, the Web service returns a list of recommended items to the native app. In contrast, for the `unfollow_shop` feature in *Jd* and the `cancel_collection` feature in *Gewala movie*, the Web app includes more cookie fields in the request header than the native app to keep states with the server.

Response Time. Usually, the `DELETE` operation can be executed very fast in both types of apps. An exception is the native app of *Amazon Shopping* that transfers data by HTTPS protocol, and spends non-negligible time on HTTPS handshaking.

Energy Consumption. Although it is observed that the two features in *Amazon Shopping* consume more traffic volume and have more response time in its native app, the native app consumes less energy. It indicates that we cannot determine the energy consumption of an application by only the data traffic and response time.

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

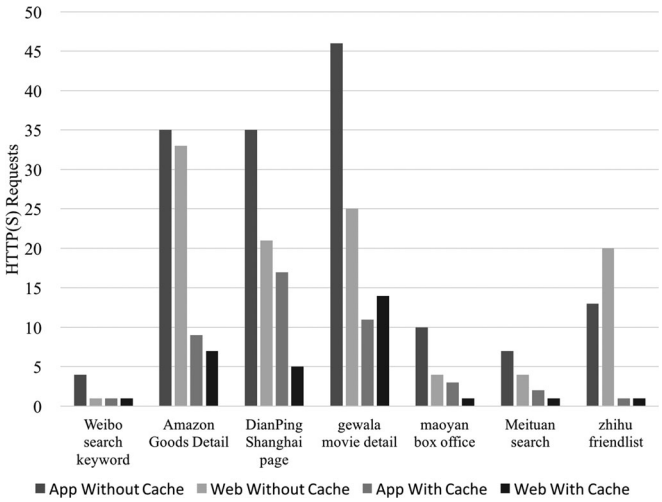


Fig. 12. HTTP(S) requests in GET.

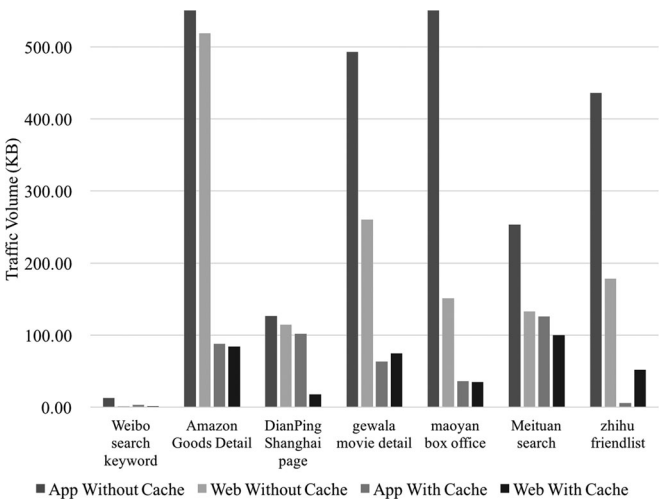


Fig. 13. Traffic volume in GET.

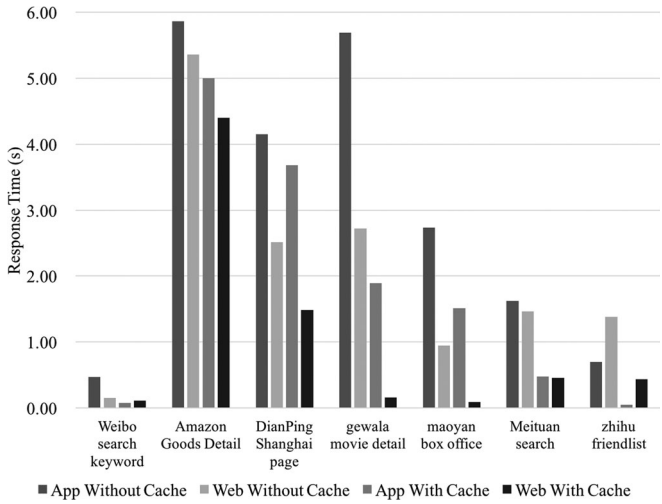


Fig. 14. Response time in GET.

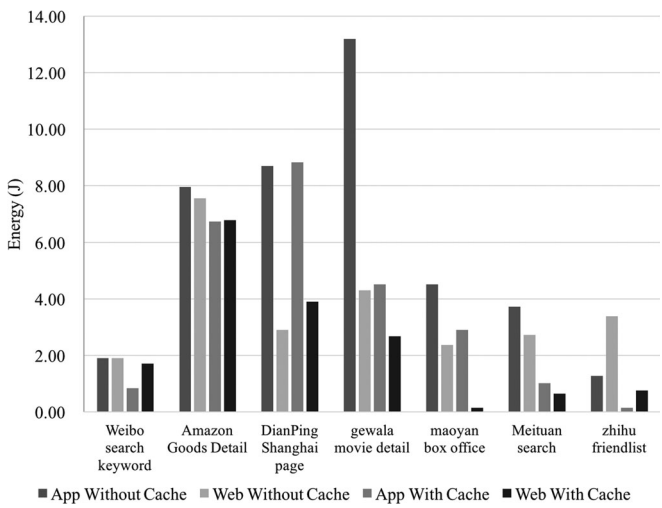


Fig. 15. Energy consumption in GET.

5.3 GET

In the semantics of RESTful Web services, `GET` refers to the request that retrieves a representation of a resource from the Web server. We examine seven typical features that need `GET` operations, i.e., the `search_keywords` feature in *Weibo*, the `shanghai_page` feature in *DianPing*, the `goods_detail` feature in *Amazon Shopping*, the `movie_detail` feature in *Gewala movie*, the `box_office` feature in *Maoyan movie*, the `search` feature in *Meituan*, and the `friendlist` feature in *Zhihu*. Figs. 12, 13, 14, and Fig. 15 show the number of HTTP(S) requests, the traffic volume, the response time, and the energy consumption in `GET` operations, respectively.

Number of Requests. In the `GET` operations, the Web apps usually initiate more HTTP(S) requests than the native apps for the same feature. It is because the Web app needs to download all the needed resources, including JavaScript files, CSS files, and other resources. However, for the `search_keywords` feature in *Weibo*, the `movie_detail` feature in *Gewala movie*, etc., the native apps initiate more requests than the Web apps. For example, we find that the native app receives the friends' information as well as the avatars according to the key words in *Weibo*, and fetches pictures of higher quality to provide better user experience in *Gewala movie*. Such a difference is also caused by the different semantics of targeted Web services. As described in Fig. 12, the Web app of *Zhihu* fetches all the friends' avatars, but the native app fetches only a part of friends' avatars. In addition, apps with cache can save up to 80 percent requests. With cache enabled, the number of HTTP(S) requests in both native apps and Web apps can be significantly reduced.

Traffic Volume. The native app of *Weibo* consumes much more network traffic than the Web app because it requests to download extra users' avatars. We have mentioned that the Web app of *Zhihu* fetches all friends' avatars. However, we are surprised to find that the Web app consumes less network traffic than the native app without cache. We explore the raw traffic data, and find that the native app fetches avatars of high-quality portraits. In contrast, the Web app fetches avatars of low-quality portraits. *Generally, the cache mechanism in `GET` operations is quite efficient*. For most features, both native apps and Web apps with cache enabled can consume less network traffic. The native app of *Google Map* can save as much as 90.9 percent traffic volume when the cache is enabled. However, the native app of *Baidu Map* disables cache for most resources, so the traffic volume reduces quite marginally. When we consider the network traffic in the with-cache situation, *it is surprising to find that*

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

the performance of some Web apps may be better than the native ones. The Web app of *Maoyan movie* takes advantage of `AppCache` [23], an advanced feature provided by HTML5 allowing the developers to configure stable resources rather than relying on only the browsers. In a sense, the resources are configured at finer granularity so a lot of duplicate transfers are avoided.

Response Time. Compared to the other three operations, the response time of `GET` operations should be more complex, as the smartphone needs to fetch a number of resources from the server and thus requires more network bandwidth. The Web apps rely on the browser, where only limited number of simultaneous connections are available. For example, in FireFox and Chrome browser, only 6 concurrent connections are allowed per domain [15]. In contrast, the native apps can fully use the TCP connection pool with the supports of the underlying OS (e.g., by `AndroidHttpClient` API) or other third-party libraries such as `Volley` [22]. The performance of "*with-cache*" is always better than that of "*without cache*". In addition, sometimes the performance of Web apps with cache can be quite close to that of native apps when the cache is enabled. In particular, the Web app of *Maoyan movie* (with cache) reduces a lot of response time, and performs better than the native app for the `box_office` feature (almost 85 percent improvement).

Energy Consumption. On the whole, native apps consume less energy than Web apps for most features, but the differences become smaller when cache is enabled. With the cache enabled. Both native apps and Web apps send less requests, and reduce the cost of parsing resources, resulting in less energy consumption. For example, some pictures are loaded into the memory rather than decoded just-in-time. In the meantime, there are some features where native apps consume more energy than Web apps. For example, the native app of *DianPing* frequently records the state of smartphones and uploads data to the remote servers. Such a process would consume more energy. Another example is the `goods_detail` feature in *Amazon Shopping*. The native app employs HTTPS rather than HTTP as the communication protocol, which introduces extra handshaking phase and data decryption, thus leading to more energy consumption [20].

6 Exploring the Findings

The preceding results evidence that Web apps have a considerable proportion of performance advantages than native apps. We need to find out why or why not the Web apps' performance can reach or even exceed native apps. Considering the architectural comparison, we organize the findings and explore the possible reasons from the two aspects of the smartphone, *cache performance* and *network performance*.

6.1 Impact of Cache Performance

It is clear that the cache mechanism can improve the performance of both native apps and Web apps, for the `GET` operations.

The native apps usually transfer less data than Web apps, as the number and size of downloaded resources can be programmatically controlled by developers. The native app developers can configure which resources should be cached locally by themselves, and define when the resources need to be updated. A lot of resources such as icons, pictures, and layout configurations, are usually stored at the client side. We observe that the cache mechanism is disabled for many resources in *Baidu*, as the app intends to show the latest data.

The cache mechanism of Web apps completely relies on the browser. However, according to our previous work [6], the browser cache mechanism is far from satisfying for mobile Web apps, due to conservative expiration time, redundant transfer, and same content with different URLs that are dynamically generated by JavaScript. We examine the cache configurations such as `CacheControl` field, `Expires` field, `Etag` field of the Web apps. Most of the Web apps configure the HTML page with "`Cache-Control:no-cache (store)`". Such configuration may be too coarse-grained, as all resources in the main HTML body have to be re-fetched even though they are still valid on the client side. When the cache policies are carefully configured at finer granularity, e.g., for different texts and images, the performance of Web apps can surprisingly reach or even exceed that of native apps. For example, the `search_a_person` feature in *Facebook*, the `good_detail` feature in *Amazon Shopping*, and the `View_checkboard` feature in *DianPing* configure different expiration time for the icons, small images, and the main HTML, respectively. In this way, only the need-to-update resources are downloaded when the Web apps are revisited.

6.2 Impact of Network Performance

Generally, the native apps expectedly perform better than Web apps, but proportional exceptions cannot be ignored. Indeed, the performance variety can significantly derive from the programming languages (i.e., Java V. S. HTML/JavaScript), but it is out of the topic of this article. Instead, we focus on the factors that may impact the network performance, i.e., the connection management, the libraries, and the protocols.

6.2.1 Impact of Connection Management

First, we focus on the network connections management of the smartphone, as it determines how many resources can be used by the apps, and could be a main factor affecting the performance. We employ the techniques proposed in [16] to analyze the TCP connections from the WireShark logs.

As mentioned above, the Web apps can use at most 6 concurrent TCP connections per domain, while the native apps can fully explore the TCP connections to reach better performance. For example, by using the `AndroidHttpClient` API and other system-level libraries such as `android.net.http`, `org.apache.http`, and `java.net.Socket`, or using third-party libraries such as `Volley` [22] (20 concurrent connections at maximum) and `android-async-http` library [24] (10 concurrent connections at maiximum), developers can configure the "`Connection:Keep-Alive`" in the request header or set "`setReuseAddress(true)`" to allocate more sockets. In practice, we observe that the number of concurrent TCP connections of native apps can be quite flexible (varying from 10 to 20), but always significantly more than the browsers.

However, although the TCP connections cannot be expanded due to the browser constraints, it is argued that the reuse of connections can be a promising option [14]. For some features provided by *Facebook* and *Google Map*, the WireShark traces indicate that the Web apps can reuse

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

existing TCP connections to reduce the latency of fetching new resource objects, rather than initializing a new connection. In this way, the performance such as response time and energy could be improved. However, it requires the server-side protocol support to enable the reuse of TCP connections, e.g., both *Google* and *Facebook* enable SPDY [25].

6.2.2 Impact of Transfer Protocols

From our observation, the choice of protocol can be quite diverse among service providers in their apps. Three providers, i.e., *Facebook*, *Google Map*, and *Douban*, use HTTPS in both native apps and Web apps. The providers such as *Weibo*, *Dianping* use HTTP in both native apps and Web apps. In contrast, the *Amazon Shopping* and *Zhihu* use HTTPS in their native apps, but use HTTP in the Web apps. Hence, we focus on these two service providers.

We find that the Web apps can beat the native apps for these two service providers in almost 36.6 percent genres. More specifically, the Web apps can beat native apps in almost all `PUT`, `POST`, and `DELETE` operations. It implies that the HTTPS can introduce overhead in some cases. When exploring the traces, we find that the size of HTTP messages is usually not very large, i.e., less than 50 KB. From the trace of Fiddler and WireShark, such overhead mainly comes from the SSL handshake phase. The overhead ranging from 4.32 KB to 11 KB (mean: 8.11 KB, median: 7.4 KB) depends on several factors including the certificate size. In addition, HTTPS cost extra energy for message encryption and decryption. In this way, *we can infer that, for small-size message, HTTPS can introduce more network payload, especially the handshake-phase overhead.*

7 Implications and Suggestions

Based on the findings and analysis, we then move to *RQ3*, i.e., *What implications can be leveraged to make Web apps and native apps provide comparable performance*? We make analysis to understand the bottlenecks and suggest some recommendations to improve the performance of mobile apps on smartphones, e.g., making the Web apps reach comparable performance with native apps.

- *Supporting fine-granularity cache management.* Undoubtedly, the size of resources fetched from the Web servers significantly impacts the performance of client-side apps. The service providers and the app developers need to determine more carefully which resource objects should be fetched when the app is revisited. The native apps provide developers very flexible options to configure the cached resources, while the Web apps heavily rely on the browser. However, the cache management of Web apps is not satisfactory when requesting Web services [26], due to the too coarse-grained configurations of the whole HTML body (`Cache-Control:no-cache(store)`). For the cases when Web apps beat the native apps (i.e., the `Maoyan`), we find that the cache policies of resources are finely configured at the object level. A substantial proportion of resources is rather static, while quite few vary through different visits. Such a fine-grained configuration is somewhat similar to the native apps. To support such fine-grained configuration, both Web service providers and Web apps developers should focus on the cache mechanism, since better cache performance requires the cooperation of client-side code, browser runtime, and server-side support. Some recent features such as HTML5 `AppCache` and `localstorage` can be leveraged to configure the expiration time for stable resources. Besides the new features, another solution that can be applied to existing Web apps is to separate the HTML into two parts, in order to load the changing resources dynamically and add them to the page on demand. We have made several efforts on improving the cache management of mobile Web apps [27], [28].
- *Reducing HTTPS overhead.* Although the HTTPS protocol becomes popular, the additional overhead of data traffic should be addressed, especially for those small-sized HTTP requests. To make the tradeoff between security and performance, some possible approaches can be leveraged to mitigate the overhead. We suggest that the providers should use lightweight SSL handshakes. Specifically, Session Identifier and Session Ticket can be used to resume previously terminated SSL sessions with low handshake overhead [20], [29]. Session ticket is preferred because the SSL state is stored at the client side.
- *Exploring connection reuse for Web apps.* The Web apps may suffer from the limited number of concurrent TCP connections. However, in some cases, we find that the browser can reuse TCP connections of previously requested resources to avoid the initialization cost for a new request resource, and the performance seems to be improved. Recently emerging protocols such as SPDY [25] and HTTP/2 [30] support the reuse of existing connections. In addition, we are aware that a lot of popular Web service providers and browsers (Chrome, FireFox, and Safari) begin to employ these new protocols.
- *Avoiding unnecessary complexities.* We observe that sometimes the design and implementation of both kinds of apps are not perfect enough so that both could be improved. For some `POST`, `PUT`, and `DELETE` operations, the native apps may perform more poorly than Web apps, e.g., the `collect_goods` feature in *Amazon*, the `delete_timeline` feature in *Weibo*, and the `delete_cart` feature in *Jd*. The reason is that Web services providers enforce their APIs to return extra data for refreshing the user interfaces. However, such response data may not be always necessary. Web service providers should give the Web developers alternatives to programmatically filter the response data. Besides, in the Web app of *Jd*, the `add_goods_to_cart` feature receives the same data in the "*cart*" and "*cartJson*" fields. Such unnecessary redundancy should be carefully avoided.
- *Unifying RESTful API semantics.* In some cases, it is observed that RESTful Web services have different semantics for native apps and Web apps given the same feature. It indicates that service providers have to maintain different server-side support for the two client-side apps, although the apps access the same back-end logic and data. When the app developers need to implement both clients, they have to understand different server-side API semantics, respectively. Hence, service providers had better uniformly organize the RESTful APIs to reduce the cost of development and maintenance for developers.

8 Threats to Validity

We conducted our empirical analysis of only 328 distinct Web services from 12 typical service providers. The selected

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

service providers cover various domains such as `Map`, `Social Network`, `e-Commerce`, and `Entertainment`. In addition, these service providers come from both global (i.e., *Google*, *Facebook*, *Amazon*) and local (i.e., *Baidu*, *Jd*, *Zhihu*, and *Dianping*). In this way, the diversity of service providers can reduce the selection bias of our empirical study. Our established experimental framework and trace-based analysis approach can be generalized to other Web services.

Nevertheless, considerable care and attention should be addressed to ensure the rigor of this measurement study. However, as with any chosen research methodology, it is hardly without limitations. In this section, we discuss the potential threats for validating our results.

8.1 Performance and User Experience

Our measurement study in this article mainly focuses on the performance at the network level and energy consumption of native apps and Web apps, given the same requested functionality. We employ the number of requests, traffic volume, and response time to quantify the performance. Indeed, the performance of apps has much broader meanings that are usually related to user experience, including look and feel, interaction patterns, context awareness, personalization, and so on. However, many aspects of user experience cannot be directly characterized by quantitative metrics. Therefore, the principle of our study is to compare the relatively objective metrics when the native apps and Web apps executing the same user-perceived functionalities that have exactly the same contents and similar UIs displayed to users on native apps and Web apps.

In addition, native apps may implement a certain smartness such as context awareness and energy awareness, making the comparison unfair. To alleviate such an issue, we ensure a stable context for the smartphones which are connected to a high-speed WiFi and the battery energy is always at 100 percent level (with adaptor charging). Such a stable context could disable the smartness of native apps to some extent but may not totally remove it. We plan to comprehensively study the smartness of native apps and their influences on user experience in our future work.

8.2 Generalization of Findings

The most considerable concern of our empirical study is the validity of findings on the native and Web apps from other service providers that are not covered in this article. It is true that the vendor-specific implementation of apps can have impacts on the performance. However, our results can be meaningful for end-users to select the proper channel for accessing the services, and for developers to fix the potential imperfect designs or even performance bugs, such as redundant response, coarse-grained cache configurations, and so on.

Additionally, some of our results can be generalized to the service providers when they design their apps. For example, we observe that the fine-granularity cache configuration of objects in Web apps can significantly improve the performance, especially for reducing the payloads of large-size but rather stable resources such as HTML body and images. Instead of using the default heuristic cache configuration, the Web apps distinguish the configurations of the images of goods and those of logos, footers, and icons.

8.3 Versions of Apps

Another threat for reproducing our results is the changing versions for apps. To overcome this threat, we download the `.apk` files of the versions of native apps evaluated in our article. Practitioners could use these files to reproduce the experiments described in this article. However, the Web apps cannot be treated in the same way. It is known that the Web apps always deliver the latest version to end-users. Although we can download the `.html` files of Web apps and request them from our server, some dynamic resources generated by JavaScript cannot be fully covered. In this way, it can be hard to completely reproduce the results of Web apps.

8.4 Heterogeneous Platforms

What is worth mentioning is the platform heterogeneity. Our experiments were made on only the Android platform. However, different platforms vary in network performance by using different libraries for the native apps. For example, it is reported that the HTTP processing is much better on iOS platform than that on other platforms [16]. Our approach relies on a proxy intercepting HTTP-level traces for Android devices. However, this tool cannot work for iOS devices.

The selected browser used in our experiment is only Google Chrome, as Chrome is quite popular and its kernel takes into account cross-platform issues. It is quite possible that different browsers may perform variously for the same Web app. To overcome this threat, we can reproduce our experiments on the combinations of different versions of Android platforms and different Web browsers such as Chrome, FireFox, and Opera.

9 Related Work

In this article, we study the performance of native apps and Web apps considering the same functionalities delivered through Web services. Web services research on desktop computing has been well surveyed by a lot of literatures [31], [32]. With the wide adoption of mobile devices such as smartphones and tablet computers, the application context of Web services has evolved from the desktop computing to the mobile computing. However, research on the usage of Web services on mobile devices is just at the beginning.

Srirama et al. [33] and Wagh et al. [34] focused on the Web services provisioning techniques for mobile devices. Guinard et al. [35] suggested using Device Profile for Web Services (*DPWS*) and REST to easily integrate physical devices into existing enterprise information systems as Web service providers. However, it is much more common for mobile devices acting as Web services consumers instead of Web services providers.

Although Tekli et al. [36] focused on SOAP performance enhancement, RESTful Web services are considered to be a more promising solution on mobile devices that have relatively limited computation resources. In practice, it is reported that most major service providers have adopted RESTful fashion [34], [37]. Wagh et al. [34] made a comparison study on the performance evaluation on handheld devices. Arroqui et al. [38] showed that RESTful Web Services improve the efficiency of data transfer for mobile devices. Hamad et al. [39] showed that RESTful Web services consumed less message size and response time than SOAP Web services for mobile devices. Fokaefs et al. [40],

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

KamalDldin et al. [41], and Mizouni et al. [42] focused on the performance of Web services on mobile platforms, and demonstrated that the RESTful pattern is much more suitable for resources-constrained mobile devices.

Due to the benefit of cross-platform accessibility by Web apps, some efforts have been made to improve the performance. Jamal et al. [43] used a cloud-based proxy and cache mechanism to enhance the performance of RESTful Web services for mobile devices. Agabaov et al. [44] showed that Google presented Flywheel, an HTTP proxy service, to reduce the page size and response time to speed up Web apps. Wang et al. [45] and our recent work [6] focused on the cache mechanism to improve the performance of Web apps. Butkiewicz et al. [46] presented *KLOTSKI*, a system that prioritized the content to improve the user experiences.

Compared to previous efforts, our research is driven by an entirely different motivation. Either the native app and Web app has its own advantages and disadvantages. Although there have been a long debate on native app and Web app on mobile devices [47], very few efforts have been made to understand the differences on using Web services. For the research community of Services computing, it is worth characterizing and understanding the different resource usages of native apps and Web apps for consuming given the same functionality.

To the best of our knowledge, we make the first step in characterizing the performance of Web services on smartphones by taking into account different client-side apps. Our derived knowledge on the cache management and network optimization can help developers take full advantages of prior technologies to enhance the performance and user experience of Web services on smartphones, for both native apps and Web apps. We can further help Web service providers to make the tradeoffs of which types of apps to develop, and help the end-users select which type of apps as the access channel. In particular, we believe that Web service providers can leverage our findings and explore the benefits gained from Web apps such as platform-neutrality, ease of deployment, and revenue by existing advertisements.

10 Conclusion

This article has conducted the first comparison study on the performance of native apps and Web apps from the perspective of Web services, in terms of HTTP(S) requests, traffic, round-trip time, and energy consumption. Although native apps usually performed better than Web apps, some exceptional scenarios were observed. We further explored the potential reasons and implications, so that developers can optimize their apps.

In the future, there are two main ongoing efforts. First, we plan to make "What-If" measurement over the existing dataset. We plan to compare the different protocols and cache management to explore the ideal upper bound of performance optimization. Such a measurement study can help to have more insights of Web services utilization on smartphones.

Second, we plan to provide some browser-level optimizations for improving Web apps. There are three alternatives to try against. The first one is made at the resource-level for reducing data transfer of Web services. We aim to optimize cache configurations based on experiences from our previous study [6], by reducing the redundant transferred resources and same-content resources. The second one is at the application-level. We aim to leverage the *localStorage* [48] and application caching mechanism (*AppCache*) [23], which take effect at fine granularity of resource items rather than complete list of resources, hence some data could be loaded locally instead of transferred by Web services. The last one is at the protocol-level. We also plan to evaluate the latest HTTP/2 [30] protocol in terms of data compression and reuse of single TCP for the dataset of this article.

Acknowledgments

This work was supported by the National Key Research and Development Program of China under Grant No. 2016YFB1000105, the Natural Science Foundation of China (Grant No. 61370020 and 61421091), and the Microsoft-PKU Joint Research Program. Yun Ma and Xuanzhe Liu are co-primary authors of this work.

References

[1] T. Petsas, A. Papadogiannakis, M. Polychronakis, E. P. Markatos, and T. Karagiannis, "Rise of the planet of the apps: A systematic study of the mobile app ecosystem," in *Proc. Internet Meas. Conf.*, 2013, pp. 277–290.

[2] N. Serrano, J. Hernantes, and G. Gallardo, "Mobile web apps," *IEEE Softw.*, vol. 30, no. 5, pp. 22–27, Sep./Oct. 2013.

[3] Hybrid apps. [Online]. Available: https://en.wikipedia.org/wiki/Multi-channel_app_development, Accessed in: 2016.

[4] I. J. M. Ruiz, M. Nagappan, B. Adams, T. Berger, S. Dienst, and A. E. Hassan, "Analyzing ad library updates in android apps," *IEEE Softw.*, vol. 33, no. 2, pp. 74–80, Mar./Apr. 2016.

[5] H. Li, et al., "Characterizing smartphone usage patterns from millions of android users," in *Proc. Internet Meas. Conf.*, 2015, pp. 459–472.

[6] Y. Ma, X. Liu, S. Zhang, R. Xiang, Y. Liu, and T. Xie, "Measurement and analysis of mobile web cache performance," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 691–701.

[7] Y. Liu, et al., "Characterizing restful web services usage on smartphones: A tale of native apps and web apps," in *Proc. IEEE Int. Conf. Web Ser.*, 2015, pp. 337–344.

[8] Apache cordova, apache cordova is a platform for building native mobile applications using html, CSS and javascript. [Online]. Available: https://cordova.apache.org/, Accessed in: 2016.

[9] Manifoldjs, the simplest way to create hosted apps across platforms and devices. [Online]. Available: http://www.manifoldjs.com/, Accessed in: 2016.

[10] React native, a framework for building native apps using react. [Online]. Available: https://facebook.github.io/react-native/, Accessed in: 2016.

[11] W3C, web services activity. [Online]. Available: http://www.w3.org/2002/ws/, Accessed in: 2016.

[12] C. Pautasso, O. Zimmermann, and F. Leymann, "Restful web services versus big'web services: Making the right architectural decision," in *Proc. 17th Int. Conf. World Wide Web*, 2008, pp. 805–814.

[13] RFC 2616, hypertext transfer protocol. [Online]. Available: http://www.w3.org/Protocols/rfc2616/rfc2616.html, Accessed in: 2016.

[14] X. S. Wang, A. Balasubramanian, A. Krishnamurthy, and D. Wetherall, "How speedy is SPDY?" in *Proc. 11th USENIX Symp. Netw. Syst. Des. Implementation*, 2014, pp. 387–399.

[15] X. S. Wang, A. Balasubramanian, A. Krishnamurthy, and D. Wetherall, "Demystifying page load performance with WProf," in *Proc. 10th USENIX Symp. Netw. Syst. Des. Implementation*, 2013, pp. 473–486.

[16] A. A. Sani, et al., "The wireless data drain of users, apps, & platforms," *Mobile Comput. Commun. Rev.*, vol. 17, no. 4, pp. 15–28, 2013.

[17] F. Qian, Z. Wang, A. Gerber, Z. M. Mao, S. Sen, and O. Spatscheck, "Characterizing radio resource allocation for 3G networks," in *Proc. 9th ACM SIGCOMM Conf. Internet Meas.*, 2010, pp. 137–150.

[18] F. Xu, Y. Liu, Q. Li, and Y. Zhang, "V-edge: Fast self-constructive power modeling of smartphones based on battery voltage dynamics," in *Proc. 10th USENIX Symp. Netw. Syst. Des. Implementation*, 2013, pp. 43–55.

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.

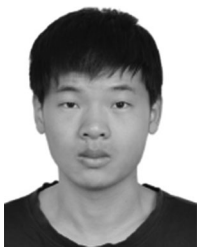
[19] Wireshark. [Online]. Available: https://www.wireshark.org/, Accessed in: 2016.
[20] F. Qian, S. Sen, and O. Spatscheck, "Characterizing resource usage for mobile web browsing," in *Proc. 1st Int. Conf. Mobile Syst. Appl Serv.*, 2014, pp. 218–231.
[21] Okhttp. [Online]. Available: http://square.github.io/okhttp/, Accessed in: 2016.
[22] Volley, an http library that makes networking for android apps easier and most importantly, faster. [Online]. Available: https://developer.android.com/training/volley/index.html, Accessed in: 2016.
[23] Using the application cache. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Using_the_application_cache, Accessed in: 2016.
[24] Android asynchronous http client, a callback-based http client library for android. [Online]. Available: http://loopj.com/android-async-http/
[25] SPDY protocol - draft 3. [Online]. Available: http://www.chromium.org/spdy/spdy-protocol/spdy-protocol-draft3, Accessed in: 2016.
[26] X. Liu, Y. Ma, Y. Liu, T. Xie, and G. Huang, "Demystifying the imperfect client-side cache performance of mobile web browsing," *IEEE Trans. Mobile Comput.*, vol. 15, no. 9, pp. 2206–2220, Sep. 2016.
[27] X. Liu, Y. Ma, S. Dong, Y. Liu, T. Xie, and G. Huang, "ReWAP: Reducing redundant transfers for mobile web browsing via app-specific resource packaging," *IEEE Trans. Mobile Comput.*, vol. 16, no. 9, pp. 2625–2638, Sep. 2017.
[28] X. Liu, Y. Ma, X. Wang, Y. Liu, T. Xie, and G. Huang, "SWAROVsky: Optimizing resource loading for mobile web browsing," *IEEE Trans. Mobile Comput.*, vol. 16, no. 10, pp. 2941–2954, Oct. 2017.
[29] The IETF TLS protocol version 1.0. [Online]. Available: https://www.ietf.org/rfc/rfc2246.txt, Accessed in: 2016.
[30] HTTP/2, what is HTTP/2? [Online]. Available: http://http2.github.io/, Accessed in: 2016.
[31] S. Wang, W. A. Higashino, M. Hayes, and M. A. Capretz, "Service evolution patterns," in *Proc. IEEE Int. Conf. Web Ser.*, 2014, pp. 201–208.
[32] O. Kondratyeva, A. Cavalli, N. Kushik, and N. Yevtushenko, "Evaluating quality of web services: A short survey," in *Proc. IEEE 20th Int. Conf. Web Ser.*, 2013, pp. 587–594.
[33] S. N. Srirama, M. Jarke, and W. Prinz, "Mobile web service provisioning," in *Proc. Int. Conf. Internet Web Appl. Ser./Adv. Int. Conf. Telecommun.*, 2006, pp. 120–120.
[34] K. Wagh and R. Thool, "A comparative study of soap versus rest web services provisioning techniques for mobile host," *J. Inf. Eng. Appl.*, vol. 2, no. 5, pp. 12–16, 2012.
[35] D. Guinard, V. Trifa, S. Karnouskos, P. Spiess, and D. Savio, "Interacting with the SOA-based internet of things: Discovery, query, selection, and on-demand provisioning of web services," *IEEE Trans. Ser. Comput.*, vol. 3, no. 3, pp. 223–235, Jul.–Sep. 2010. [Online]. Available: http://doi.ieeecomputersociety.org/10.1109/TSC.2010.3
[36] J. Tekli, E. Damiani, R. Chbeir, and G. Gianini, "SOAP processing performance and enhancement," *IEEE Trans. Ser. Comput.*, vol. 5, no. 3, pp. 387–403, Jul.–Sep. 2012.
[37] J. Huang, Q. Xu, B. Tiwana, Z. M. Mao, M. Zhang, and P. Bahl, "Anatomizing application performance differences on smartphones," in *Proc. 8th Int. Conf. Mobile Syst. Appl. Ser.*, 2010, pp. 165–178.
[38] M. Arroqui, C. Mateos, C. Machado, and A. Zunino, "Restful web services improve the efficiency of data transfer of a whole-farm simulator accessed by android smartphones," *Comput. Electron. Agriculture*, vol. 87, pp. 14–18, 2012.
[39] H. Hamad, M. Saad, and R. Abed, "Performance evaluation of restful web services for mobile devices," *Int. Arab J. e-Technol.*, vol. 1, no. 3, pp. 72–78, 2010.
[40] M. Fokaefs, R. Mikhaiel, N. Tsantalis, E. Stroulia, and A. Lau, "An empirical study on web service evolution," in *Proc. IEEE Int. Conf. Web Ser.*, 2011, pp. 49–56.
[41] K. Mohamed and D. Wijesekera, "Performance analysis of web services on mobile devices," in *Proc. 3rd Int. Conf. Ambient Syst. Netw. Technol.*, 2012, pp. 744–751.
[42] R. Mizouni, M. A. Serhani, R. Dssouli, A. Benharref, and I. Taleb, "Performance evaluation of mobile web services," in *Proc. 9th IEEE Eur. Conf.*, 2011, pp. 184–191.
[43] S. Jamal and R. Deters, "Using a cloud-hosted proxy to support mobile consumers of restful services," *Procedia Comput. Sci.*, vol. 5, pp. 625–632, 2011.
[44] V. Agababov, et al., "FlyWheel: Google's data compression proxy for the mobile web," in *Proc. 12th USENIX Symp. Netw. Syst. Des. Implementation*, 2015, pp. 367–380.
[45] X. S. Wang, A. Krishnamurthy, and D. Wetherall, "How much can we micro-cache web pages?" in *Proc. Conf. Internet Meas. Conf.*, 2014, pp. 249–256.
[46] M. Butkiewicz, D. Wang, Z. Wu, H. V. Madhyastha, and V. Sekar, "KLOTSKI: Reprioritizing web content to improve user experience on mobile devices," in *Proc. 12th USENIX Symp. Netw. Syst. Des. Implementation*, 2015, pp. 439–453.
[47] Z. Hemel and E. Visser, "Declaratively programming the mobile web with mobl," *ACM SIGPLAN Notices*, vol. 46, no. 10, pp. 695–712, 2011.
[48] The localstorage attribute of html5. [Online]. Available: https://html.spec.whatwg.org/#the-localstorage-attribute, Accessed in: 2016.



Yun Ma received the BS and the PhD degrees from the School of Electronics Engineering and Computer Science, Peking University, in 2011 and 2017, respectively. He is now working as a postdoc in the School of Software, Tsinghua University, Beijing, China. His research interests include web engineering and mobile computing. He is a student member of the IEEE.



Xuanzhe Liu received the PhD degree from Peking University, in 2009. He is an associate professor in the School of Electronics Engineering and Computer Science, Peking University, Beijing, China. His research interests include the area of services computing, mobile computing, and Web engineering. He is a member of the IEEE.



Yi Liu is now working toward the PhD degree in the School of Electronics Engineering and Computer Science, Peking University, Beijing, China. His research interests include mobile computing, network systems, and Web engineering.



Yunxin Liu received the PhD degree in computer science from the Shanghai Jiao Tong University, in 2011 (through the SJTU-MSRA joint PhD program). He is a lead researcher at Microsoft Research Asia. His current research interests include mobile systems and networking. He is a member of the IEEE.



Gang Huang received the PhD degree from the School of Electronics Engineering and Computer Science, Peking University, Beijing, China, 2003. He is now a full professor in the Institute of Software, Peking University. His research interests include the area of middleware of cloud computing and mobile computing. He is a member of IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: University of Wolverhampton. Downloaded on November 17,2022 at 12:14:07 UTC from IEEE Xplore. Restrictions apply.